

## Day 8 Evidence Workbooklet

### Expected-vs-Actual Debugging and Test Design

Engineering practice → observed branch outcome → expected branch outcome → likely cause → regression test

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

|       |         |         |               |                                     |  |
|-------|---------|---------|---------------|-------------------------------------|--|
| Name  |         | Section |               | Date                                |  |
| Score | ____/34 |         | Mastery check | secure / developing / needs support |  |

The sensor-kit checkout decision logic is now being tested. Some branch outcomes look safe for one kit but fail for another. Today the focus is debugging: separate the observed behavior from the expected behavior, identify a likely cause, choose a minimal fix, and keep a test that would catch the bug again.

|                           |                                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Engineering task</b>   | Use a repair log to decide whether the checkout decision logic is trustworthy before the lab uses it.                                   |
| <b>Computational move</b> | Compare expected and actual behavior, connect the mismatch to a code cause, and design tests that cover important branch outcomes.      |
| <b>Python focus</b>       | expected branch outcome, actual branch outcome, condition order, Boolean release condition, minimal fix, boundary and regression tests. |
| <b>Evidence to keep</b>   | symptom, expected branch outcome, actual branch outcome, likely cause, minimal repair, test case, and support category.                 |

### Day 8 working rules

1. A symptom says what went wrong.
2. Expected behavior says what should have happened.
3. Actual behavior says what the code did.
4. A likely cause points to a condition or order problem.
5. A regression test keeps a known bug from quietly returning.

# 1 Separate symptom, cause, and repair

recognize debug

The checkout desk finds that some kits are assigned incorrectly. A good debug note separates what happened, what should have happened, and what line likely caused it.

```
1 charge_percent = 82
2 calibration_label = "expired"
3 borrower_initials = "KM"
4
5 if charge_percent >= 80:
6     action = "release kit"
7 elif calibration_label != "current":
8     action = "calibration bench"
9 elif borrower_initials == "":
10    action = "borrower log needed"
11 else:
12    action = "hold for review"
```

| Pts. | Prompt                                                                | My evidence |
|------|-----------------------------------------------------------------------|-------------|
| 1    | What branch outcome will the current code assign?                     |             |
| 1    | What branch outcome should the kit receive if calibration is expired? |             |
| 1    | Which line lets the kit release too early?                            |             |
| 1    | Is the first problem a syntax error or a logic error?                 |             |

## 2 Build an expected-vs-actual table

[trace](#)[debug](#)

Use the same code. Complete the table and name the likely cause.

| Pts. | Case                                           | Expected branch outcome | Actual branch outcome | Likely cause |
|------|------------------------------------------------|-------------------------|-----------------------|--------------|
| 2    | charge 82, expired calibration                 | calibration bench       |                       |              |
| 2    | charge 76, current calibration                 | charge station          |                       |              |
| 2    | charge 90, current calibration, blank initials | borrower log needed     |                       |              |
| 2    | Explain which case best exposes the bug.       |                         |                       |              |

### 3 Design tests before trusting a repair

[test](#) [explain](#)

A repair is not trustworthy just because it works for one case. Pick cases that cover the branch outcomes.

| Pts. | Prompt                                                      | My test evidence |
|------|-------------------------------------------------------------|------------------|
| 2    | Give a test case that should produce "charge station".      |                  |
| 2    | Give a test case that should produce "calibration bench".   |                  |
| 2    | Give a test case that should produce "borrower log needed". |                  |
| 2    | Give a test case that should produce "release kit".         |                  |

#### Regression-test idea

A regression test is a small case you keep because it catches a bug that already happened once. If the bug comes back, that test should fail again.

4 Choose a minimal fix

implement debug

One safe repair is to require all release evidence before assigning "release kit". Complete the condition.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "hold for review"
```

| Pts. | Prompt                                                                     | My response |
|------|----------------------------------------------------------------------------|-------------|
| 2    | Why does the release condition need three evidence checks?                 |             |
| 2    | Which branch outcome will the expired-calibration case now receive?        |             |
| 2    | What risk remains if damage evidence is added later but not checked first? |             |
| 2    | Name one test you would rerun after this repair.                           |             |

## 5 Write a repair log

[debug](#) [test](#) [explain](#)

| Log field         | My repair evidence |
|-------------------|--------------------|
| Symptom           |                    |
| Expected behavior |                    |
| Likely cause      |                    |
| Minimal fix       |                    |
| Regression test   |                    |

| Pts. | Debugging note                                                                                                                                                          |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4    | Explain why expected-vs-actual evidence is stronger than saying "the code is wrong." Use one branch outcome from the sensor-kit checkout example.                       |
| 2    | Circle the support plan you would use next: symptom / expected branch outcome / actual branch outcome / cause / test case / minimal fix. Name one next action:<br>_____ |

**Bridge to Exam 1 and Day 10**

Day 9 is Exam 1. Use expected-vs-actual evidence, minimal repair, boundary tests, and support-plan notes as Exam 1 preparation. Day 10 returns to branch maps and test evidence after the exam and bridges into repeated cases and loops.

# VIP Lab Alignment

## Bridge Day 8 • Contract 16b4e542994c

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

### Why this page is here

**VIP Lab Day(s):** 4

**Activities:** 18.13, 18.14, 18.15, 18.16, 18.17

Use expected-versus-actual evidence on the current sample and transfer cells instead of debugging an obsolete wrapper.

### Open these current notebook cells

| Activity / stable cells                                    | Notebook work                 | Evidence to carry forward                                 |
|------------------------------------------------------------|-------------------------------|-----------------------------------------------------------|
| <b>18.13</b><br><small>18-13-work / 18-13-transfer</small> | <b>Compare With Tolerance</b> | Compute absolute distance between two values.             |
| <b>18.14</b><br><small>18-14-work / 18-14-transfer</small> | <b>Count High Readings</b>    | Read a supplied loop without having to design it yet.     |
| <b>18.15</b><br><small>18-15-work / 18-15-transfer</small> | <b>Lamination Fee</b>         | Translate three quantity intervals into ordered branches. |
| <b>18.16</b><br><small>18-16-work / 18-16-transfer</small> | <b>Temperature Status</b>     | Compare a measurement with a maximum.                     |
| <b>18.17</b><br><small>18-17-work / 18-17-transfer</small> | <b>Pickup Window</b>          | Calculate round-trip travel time.                         |

### Readiness check before you code

1. Choose one sample or transfer case and record expected result, actual result, likely cause, and the smallest justified repair.  
\_\_\_\_\_
2. Name one boundary pair that differs by only one unit or one small measurement change. \_\_\_\_\_

### Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those studio cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.